Fondements du Modèle Relationnel et Manipulation de Données avec SQL

Thème principal : Bases de Données Relationnelles : Modélisation, Contraintes d'Intégrité et Requêtes SQL (SELECT, WHERE, JOIN)

Synthèse exécutive : Ce cours pose les fondations des Systèmes de Gestion de Bases de Données Relationnelles (SGBDR). Il explique pourquoi la gestion par fichiers bruts (CSV, TXT) montre ses limites (redondance, anomalies de mise à jour, concurrence) et comment le modèle relationnel résout ces contraintes grâce aux notions de tables, clés primaires, clés étrangères et contraintes d'intégrité. Enfin, il détaille la syntaxe fondamentale du langage SQL d'extraction (SELECT, FROM, WHERE, INNER JOIN) en fournissant une méthodologie rigoureuse pour construire des requêtes sans erreur.

Pourquoi utiliser une Base de Données Relationnelle ?

Avant l'invention des SGBD, les applications stockaient leurs données dans des fichiers texte ou CSV séparés. Cette approche pose trois problèmes majeurs : la redondance d'information (le même client réécrit dans 50 fichiers de commande), l'incohérence (mise à jour du téléphone client à un endroit mais pas aux autres) et les accès concourants (deux utilisateurs écrivent en même temps et écrasent le fichier). Le modèle relationnel, formalisé par Edgar F. Codd, organise les données sous forme de tables liées entre elles par des clés, garantissant l'intégrité et l'accès optimisé aux données via un SGBDR comme PostgreSQL ou Oracle.

- Limites des fichiers plats (CSV/TXT) : Absence d'atomicité, redondances multiples, risque de corruption lors d'accès simultanés.
- SGBDR (Système de Gestion de Bases de Données Relationnelles) : Logiciel intermédiaire garantissant la cohérence, la sécurité et la recherche efficace des données.

Cas concret enseignant : *L'enseignant prend l'exemple d'un fichier Excel unique contenant les étudiants et leurs notes de TP. Si un étudiant change de nom de famille, il faut modifier 20 lignes. Si on oublie une ligne, la base est corrompue. Dans un SGBDR, l'étudiant est stocké dans une table 'Etudiant' unique (1 seule ligne), et la table 'Note' y fait seulement référence.*

Structure du Modèle Relationnel : Tables, Clés Primaires et Clés Étrangères

Dans une relation, chaque ligne (tuple) doit être unique. Pour garantir cette unicité, on définit une Clé Primaire (PK). Une clé primaire ne peut jamais être nulle (NOT NULL) et ne peut contenir de doublons. Pour lier deux tables (par exemple faire correspondre un Étudiant à son Département d'enseignement), on insère la clé primaire du Département dans la table Étudiant : cette colonne devient une Clé Étrangère (FK). L'intégrité référentielle interdit d'insérer un étudiant rattaché à un département qui n'existe pas.

- Relation (Table) : Ensemble non ordonné de tuples (lignes) possédant les mêmes attributs (colonnes).
- Clé Primaire (PK - Primary Key) : Attribut (ou ensemble d'attributs) permettant d'identifier de manière unique chaque tuple d'une table.
- Clé Étrangère (FK - Foreign Key) : Attribut d'une table qui référence la clé primaire d'une autre table, établissant un lien d'intégrité référentielle.

Cas concret enseignant : *Exemple du tableau blanc : Table ETUDIANT(num_etud PK, nom, prenom, #id_dept FK) et Table DEPARTEMENT(id_dept PK, nom_dept). Le hashtag '#' ou le préfixe FK indique la contrainte de référence.*

Extraction de Données en SQL : SELECT, FROM et WHERE

Le langage SQL (Structured Query Language) est un langage déclaratif : on indique au SGBD *ce que* l'on veut obtenir, et non *comment* le calculer. La structure de base est 'SELECT colonnes FROM table WHERE condition;'.

La clause WHERE utilise des opérateurs de comparaison (=, !=, <, >, <=, >=), des connecteurs logiques (AND, OR, NOT) et des prédicats spécifiques (BETWEEN, IN, LIKE, IS NULL).

- Projection (SELECT) : Choix des colonnes à afficher dans le résultat.
- Origine (FROM) : Spécification de la ou des tables interrogées.
- Restriction / Filtre (WHERE) : Condition logique (BOOLEAN) filtrant les tuples conservés.

Liaison entre Tables : La Jointure Interne (INNER JOIN)

Pour récupérer des informations réparties sur plusieurs tables (ex: le nom de l'étudiant et le nom de son département), on effectue une jointure. La syntaxe moderne ANSI préconise l'utilisation de 'FROM TableA INNER JOIN TableB ON TableA.cle_pk = TableB.cle_fk'. Sans la clause ON, le SGBD effectue un produit cartésien qui multiplie le nombre de lignes et produit un résultat incohérent.

- Produit Cartésien : Croisement de toutes les lignes de la table A avec toutes les lignes de la table B (à éviter absolument).
- Jointure Interne (INNER JOIN ... ON) : Association conditionnelle des tuples de deux tables partageant une équivalence de valeur (généralement A.PK = B.FK).

Cas concret enseignant : *L'enseignant insiste : 'N'écrivez plus de jointures dans le WHERE avec des virgules dans le FROM ! Utilisez exclusivement INNER JOIN ... ON pour le partiel de R1.05, c'est la norme moderne et propre.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'une table peut avoir plusieurs clés primaires ?
Réponse du professeur : Non. Une table possède UNE SEULE clé primaire. En revanche, cette unique clé primaire peut être composée de plusieurs colonnes (on parle alors de clé primaire composite, par exemple le couple (code_matiere, num_etud) pour la table de notes).

Question étudiant : Quelle est la différence entre WHERE et HAVING ?
Réponse du professeur : La clause WHERE filtre les lignes individuellement AVANT toute opération de regroupement. La clause HAVING filtre les groupes créés par GROUP BY APPRÈS l'application d'une fonction d'agrégation (comme SUM, AVG, COUNT).

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : ETUDIANT (num_etud PK, nom, prenom, moyenne, #id_dept FK) DEPARTEMENT (id_dept PK, nom_dept, ville) Écrire la requête SQL permettant de lister le nom, le prénom et le nom du département de tous les étudiants ayant une moyenne supérieure ou égale à 10/20, triés par nom de famille dans l'ordre alphabétique.

Démarche de résolution :
Étape 1 : Identifier les colonnes à afficher -> nom, prenom (table ETUDIANT) et nom_dept (table DEPARTEMENT). Donc SELECT E.nom, E.prenom, D.nom_dept. Étape 2 : Identifier la table principale et la jointure nécessaire -> FROM ETUDIANT E INNER JOIN DEPARTEMENT D ON E.id_dept = D.id_dept. Étape 3 : Identifier le critère de sélection (restriction) -> moyenne >= 10. Donc WHERE E.moyenne >= 10. Étape 4 : Trier les résultats -> ORDER BY E.nom ASC.

```
SELECT
    E.nom,
    E.prenom,
    D.nom_dept
FROM ETUDIANT E
INNER JOIN DEPARTEMENT D
    ON E.id_dept = D.id_dept
WHERE E.moyenne >= 10
ORDER BY E.nom ASC;
```

Résultat : La requête renvoie la liste filtrée des étudiants admis avec leur département respectif, correctement ordonnée sans doublons ni produit cartésien.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si l'on oublie la clause ON dans un INNER JOIN ?

R : Le SGBD génère une erreur de syntaxe ou produit un produit cartésien (chaque ligne de la première table est associée à toutes les lignes de la seconde), ce qui fausse les résultats.

Q : Quelle est la valeur renvoyée par la comparaison 'valeur = NULL' en SQL ?

R : Elle renvoie UNKNOWN (inconnu). Pour tester la présence d'une valeur nulle, il faut impérativement utiliser le prédicat 'IS NULL' ou 'IS NOT NULL'.

Q : Quelle est la règle d'intégrité référentielle concernant une clé étrangère ?

R : Toute valeur présente dans une colonne Clé Étrangère (FK) doit obligatoirement exister au préalable dans la colonne Clé Primaire (PK) de la table référencée (ou être NULL si la contrainte le permet).